



Computer Networks

Complete Exam Notes – Unit 3

Switching • Data Link Layer • Error Control

MAC Protocols • HDLC • PPP

With Diagrams, Explanations & Practice Q&A



Table of Contents

1. Switching Techniques

- 1.1 Circuit Switching
- 1.2 Packet Switching
- 1.3 Comparison Summary

2. Data Link Layer Design Issues

- 2.1 Services to the Network Layer
- 2.2 Framing Methods
- 2.3 Error Control
- 2.4 Flow Control

3. Error Detection and Correction

- 3.1 Error-Correcting Codes
- 3.2 Error-Detecting Codes

4. Carrier Sense Multiple Access Protocols

- 4.1 CSMA Variants
- 4.2 CSMA/CD

5. Classic Ethernet MAC Sublayer

- 5.1 Frame Format
- 5.2 Binary Exponential Backoff

6. Data Link Control Protocols

- 6.1 Simple Protocol
- 6.2 Stop-and-Wait Protocol
- 6.3 HDLC
- 6.4 PPP

7. Practice Questions & Answers

- 7.1 Theoretical Questions
- 7.2 Numerical Problems

1. Switching Techniques

From the perspective of telephone engineering, the phone system is divided into two principal parts: the **outside plant** (local loops and trunks, physically outside switching offices) and the **inside plant** (switches, inside switching offices). Two switching techniques are used in modern networks: **circuit switching** (traditional telephone system) and **packet switching** (used in computer networks and increasingly in voice-over-IP).

1.1 Circuit Switching

Core Concept

When a telephone call is placed, switching equipment seeks out a **physical path** from the caller's telephone to the receiver's telephone. This technique is called **circuit switching**.

Mechanism (see Fig. 1-1): Each rectangle represents a carrier switching office (end office, toll office, etc.). Each office has incoming and outgoing lines. When a call passes through a switching office, a physical connection is established between the incoming line and an outgoing line. The path is dedicated end-to-end for the duration of the call.

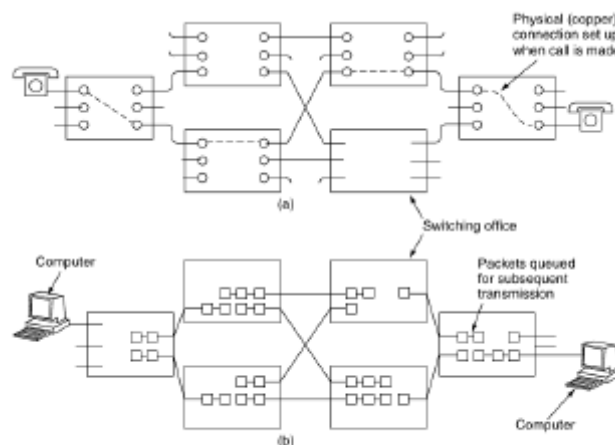


Figure 2-42. (a) Circuit switching. (b) Packet switching.

Figure 1-1: (a) Circuit switching with physical connections through switching offices. (b) Packet switching with packets queued at routers.

Historical Note: Strowger Gear

- Early connections were made manually by operators plugging jumper cables.
- **Almon B. Strowger**, a 19th-century Missouri undertaker, invented automatic circuit switching.
- **Motivation:** The other undertaker in town had his wife as the town telephone operator, who directed all funeral calls to her husband. Strowger chose to invent automatic switching rather than go out of business.
- For nearly 100 years, circuit-switching equipment worldwide was known as **Strowger gear**.

Important Properties of Circuit Switching

Property 1: End-to-end path establishment required

No data can be sent until a complete path is set up. Elapsed time between end of dialing and start of ringing: easily **10 seconds** or more for long-distance/international calls. During setup, the system hunts for a path. Call request signal must propagate to destination and be acknowledged before data transmission begins. Long setup times are undesirable for many computer applications (e.g., point-of-sale credit verification).

Property 2: Reserved path characteristics

Once setup is complete, the only delay is **propagation time** for the electromagnetic signal: approximately **5 msec per 1000 km**. No danger of congestion after call is established — no busy signals once connected. Busy signals may occur during setup due to lack of switching or trunk capacity.

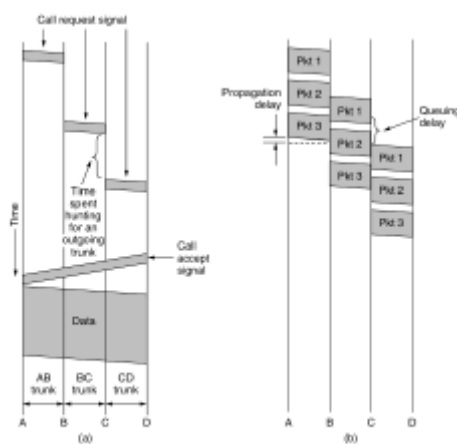


Figure 1-2: Timing of events in (a) circuit switching, (b) packet switching.

Figure 1-2: (a) Circuit switching timing: call request, trunk hunting, call accept, data transmission. (b) Packet switching timing: propagation delay and queuing delay across nodes.

1.2 Packet Switching

Core Concept

Packets are sent as soon as they are available. There is **no need to set up a dedicated path** in advance. Routers use **store-and-forward transmission**. Each packet is sent independently toward the destination. Unlike circuit switching, bandwidth is **not reserved** end-to-end.

Key Properties

1. **No fixed path** — Different packets can follow different paths depending on network conditions. Packets may arrive **out of order**.
2. **Packet size limits** — Networks place a tight upper limit on packet size. This ensures no user monopolizes a transmission line for very long, enables handling of interactive traffic, and reduces delay (first packet of a long message can be forwarded before the second one has fully arrived).
3. **Store-and-forward delay** — Accumulating a packet in router memory before forwarding exceeds the delay of circuit switching. In circuit switching, bits flow continuously through the wire.
4. **Queuing and congestion** — Packets may have to wait to be forwarded (queuing delay). Congestion occurs when many packets are sent simultaneously. No busy signal — network is always accessible, but packets may experience delay. **Congestion timing differs**: circuit switching at setup time; packet switching when packets are sent.
5. **Bandwidth efficiency** — Circuit switching: reserved but unused bandwidth is **wasted** (cannot be used by others). Packet switching: bandwidth is **not wasted** — more efficient from a system perspective. **Fundamental trade-off**: guaranteed service + wasted resources (circuit) vs. no guarantee + no waste (packet).
6. **Fault tolerance** — Packet switching is more fault tolerant — this is why it was invented. If a switch fails in circuit switching, all circuits using it are terminated. In packet switching, packets can be **routed around dead switches**.
7. **Charging algorithms** — **Circuit switching**: historically based on distance and time. **Packet switching**: connect time is irrelevant; **volume of traffic** matters. Home users: typically flat monthly rate. Backbone carriers: charge regional networks based on traffic volume.

1.3 Summary of Differences

Table 1-1: Circuit Switching vs Packet Switching

Aspect	Circuit Switching	Packet Switching
Path	Dedicated physical path	No fixed path
Setup	Required (long setup time)	None
Bandwidth reservation	Yes	No
Congestion	At setup time	During transmission
Out-of-order arrival	Impossible	Possible
Bandwidth waste	Yes (if idle)	No
Fault tolerance	Low (switch failure terminates circuits)	High (packets rerouted)
Charging basis	Distance and time	Traffic volume
Delay after setup	Propagation only	Store-and-forward + queuing

2. Data Link Layer Design Issues

The data link layer uses the physical layer's services to send and receive bits over communication channels. Its functions include: (1) providing a well-defined **service interface** to the network layer, (2) **dealing with transmission errors**, and (3) **regulating data flow** so slow receivers are not swamped by fast senders.

Encapsulation

Packets from the network layer are encapsulated into **frames**. Each frame contains: a **frame header**, a **payload field** (holds the packet), and a **frame trailer**.

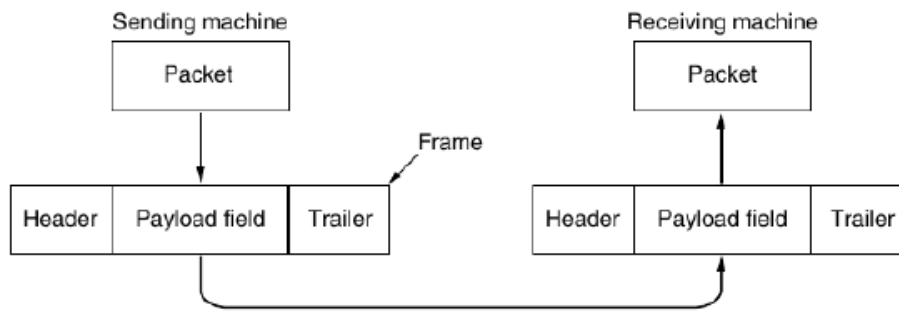


Figure 3-1. Relationship between packets and frames.

Figure 2-1: Frame structure: Header + Payload field + Trailer; packet encapsulated into frame.

2.1 Services Provided to the Network Layer

Communication Model

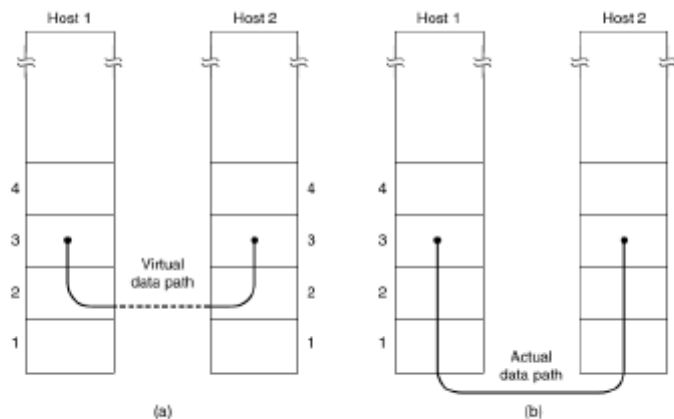


Figure 3-2. (a) Virtual communication. (b) Actual communication.

Figure 2-2: (a) Virtual communication: data link layer processes appear to communicate directly. (b) Actual communication: path goes down through layers, across physical link, up through layers.

- **Virtual communication** (Fig. 2-2a): Two data link layer processes appear to communicate directly using a data link protocol (conceptual model).
- **Actual communication** (Fig. 2-2b): The actual transmission follows a path down through the layers, across the physical medium, and up through the layers on the destination machine.

Service Type 1: Unacknowledged Connectionless Service

- Source machine sends independent frames without requiring acknowledgements.
- No logical connection established beforehand or released afterward.
- Lost frames due to noise are **not detected or recovered** at the data link layer.

- **Appropriate for:** Very low error rates (recovery left to higher layers), real-time traffic (e.g., voice) where late data is worse than bad data.
- **Example:** Ethernet.

Service Type 2: Acknowledged Connectionless Service

- No logical connections, but **each frame is individually acknowledged**.
- Sender knows whether a frame arrived correctly or was lost.
- Lost frames can be retransmitted if not acknowledged within a specified time.
- **Appropriate for:** unreliable channels (e.g., wireless systems).
- **Example:** 802.11 (WiFi).

Important Note on Acknowledgements

Providing acknowledgements in the data link layer is an **optimization**, never a requirement. The network layer could always implement its own acknowledgement and retransmission. However, the data link layer knows link-specific parameters (maximum frame length, propagation delays) that the network layer does not. If a large packet is broken into 10 frames and 2 are lost, network-layer retransmission would be very slow. Individual frame acknowledgement and retransmission correct errors more directly and quickly.

Service Type 3: Acknowledged Connection-Oriented Service

- Most sophisticated service.
- Source and destination establish a **connection** before data transfer.
- Each frame is **numbered**.
- **Guarantees:** Each frame sent is received; each frame received **exactly once**; all frames received in the **right order**.
- Provides the network layer with the equivalent of a **reliable bit stream**.
- **Appropriate for:** long, unreliable links (e.g., satellite channels, long-distance telephone circuits).

Connection-oriented transfer phases:

1. **Connection establishment:** both sides initialize variables and counters to track received frames.
2. **Data transmission:** one or more frames are transmitted.
3. **Connection release:** variables, buffers, and resources are freed.

2.2 Framing

The physical layer delivers a raw bit stream that is not guaranteed to be error-free. The data link layer must: break the bit stream into **discrete frames**, compute a **checksum** for each frame, include the checksum in the transmitted frame, and recompute and verify the checksum at the destination.

Challenge

Breaking the bit stream into frames is non-trivial. A good design must make it easy for the receiver to find the start of new frames while using minimal channel bandwidth.

Method 1: Byte Count

A field in the header specifies the number of bytes in the frame. The destination sees the byte count and knows how many bytes follow, locating the end of the frame.

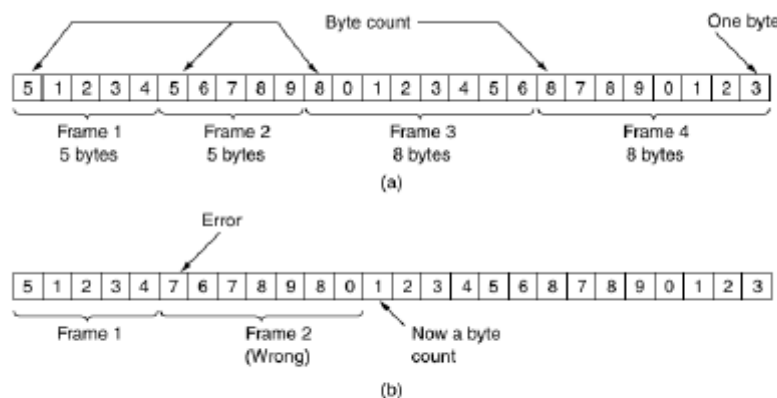


Figure 3-3. A byte stream. (a) Without errors. (b) With one error.

Figure 2-3: (a) Byte stream without errors: four frames of sizes 5, 5, 8, and 8 bytes. (b) With one error: the count can be garbled, causing loss of synchronization.

Vulnerability of Byte Count

The count can be garbled by transmission errors. If the byte count becomes corrupted, the destination loses synchronization and cannot locate the correct start of the next frame. Even if checksum detects the error, the destination cannot tell where the next frame starts. Retransmission requests are ineffective. **Rarely used by itself.**

Method 2: Flag Bytes with Byte Stuffing

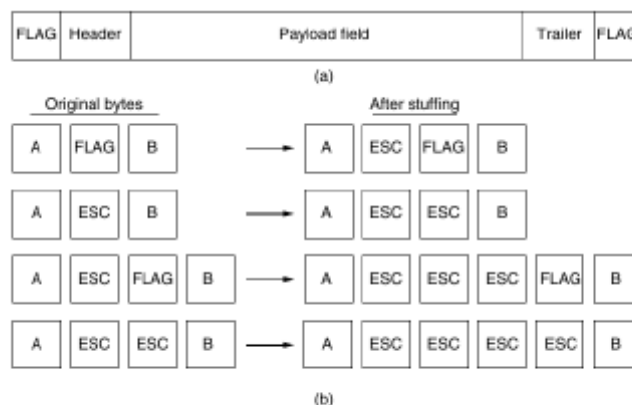


Figure 3-4. (a) A frame delimited by flag bytes. (b) Four examples of byte sequences before and after byte stuffing.

Figure 2-4: (a) Frame delimited by flag bytes. (b) Four examples of byte sequences before and after byte stuffing.

Mechanism: Each frame starts and ends with a special **flag byte**. Two consecutive flag bytes indicate the end of one frame and the start of the next. If the receiver loses synchronization, it searches for two flag bytes to resynchronize.

Problem: Flag bytes may occur in the data (e.g., binary data like photographs or songs).

Solution — Byte Stuffing: The sender's data link layer inserts a special **escape byte (ESC)** before each accidental flag byte in the data. A framing flag byte is distinguished from one in the data by the absence or presence of an escape byte before it. Receiver removes escape bytes before giving data to the network layer.

Table 2-1: Byte Stuffing Examples

Original bytes	After stuffing
A, FLAG, B	A, ESC, FLAG, B
A, ESC, B	A, ESC, ESC, B
A, ESC, FLAG, B	A, ESC, ESC, ESC, FLAG, B
A, ESC, ESC, B	A, ESC, ESC, ESC, ESC, B

Note: The byte-stuffing scheme shown is a slight simplification of the one used in **PPP (Point-to-Point Protocol)**.

Method 3: Flag Bits with Bit Stuffing

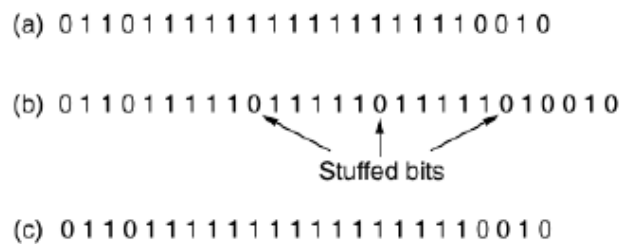


Figure 3-5. Bit stuffing. (a) The original data. (b) The data as they appear on the line. (c) The data as they are stored in the receiver's memory after destuffing.

Figure 2-5: Bit stuffing: (a) Original data. (b) The data as they appear on the line with stuffed bits. (c) The data as stored in receiver memory after destuffing.

Motivation: Byte stuffing is tied to 8-bit bytes. Framing can be done at the bit level, allowing frames with arbitrary bit counts.

Mechanism: Each frame begins and ends with the special bit pattern **01111110** (hexadecimal 0x7E). Whenever the sender encounters **five consecutive 1s** in the data, it automatically **stuffs a 0 bit** into the outgoing bit stream. This is analogous to byte stuffing but operates at the bit level. Also ensures a minimum density of transitions to help the physical layer maintain synchronization. **USB** uses bit stuffing for this reason.

Destuffing: When the receiver sees five consecutive incoming 1 bits followed by a 0 bit, it automatically **deletes** the 0 bit.

Transparency: Bit stuffing is completely transparent to the network layer. If user data contains the flag pattern 01111110, it is transmitted as 011111010 but stored in receiver memory as 01111110.

Method 4: Physical Layer Coding Violations

- Uses redundancy in physical layer encoding.
- Example: in **4B/5B** line code, 4 data bits map to 5 signal bits (16 of 32 signal possibilities are unused).
- Reserved signal patterns indicate start and end of frames.
- **Advantage:** no need to stuff data; reserved signals are easy to find.

Combined Methods

Many protocols use combinations for safety. Common pattern (Ethernet, 802.11): frame begins with a **preamble** (well-defined pattern, ~72 bits for 802.11) to allow receiver preparation, followed by a **length/count field** in the header to locate the end of the frame.

2.3 Error Control

Objective

Ensure all frames are eventually delivered to the network layer at the destination **exactly once** and in the **proper order**.

Mechanisms

1. **Acknowledgements** — Receiver sends back special control frames with **positive** or **negative** acknowledgements. Positive acknowledgement: frame arrived safely. Negative acknowledgement: something went wrong, frame must be retransmitted.
2. **Timers** — Problem: hardware troubles may cause a frame to vanish completely (noise burst), or the acknowledgement may be lost. If sender waits indefinitely for an acknowledgement, the protocol **hangs forever**. **Solution**: sender starts a timer when transmitting a frame. Timer set to expire after an interval long enough for: frame to reach destination, processing at destination, acknowledgement to propagate back. If timer expires before acknowledgement arrives, the frame is **retransmitted**.
3. **Sequence Numbers** — Danger: receiver may accept the same frame multiple times and pass it to the network layer more than once. **Solution**: assign **sequence numbers** to outgoing frames. Receiver can distinguish retransmissions from originals.

2.4 Flow Control

A sender on a fast, powerful computer may transmit frames faster than a slow receiver can accept them. Even with error-free transmission, the receiver may lose frames. *Example*: A smartphone requesting a Web page from a powerful server — the server may overwhelm the phone.

Approach 1: Feedback-Based Flow Control

- Receiver sends information back to the sender: granting permission to send more data, OR telling the sender how the receiver is doing.
- Protocol contains well-defined rules about when a sender may transmit the next frame.
- Often prohibits sending until receiver grants permission (implicitly or explicitly).

Approach 2: Rate-Based Flow Control

- Protocol has a built-in mechanism limiting sender transmission rate.
- Does **not** use feedback from the receiver.
- Primarily seen in the **transport layer**.

Prevalence

Feedback-based schemes appear at both link layer and higher layers. Higher layers are more common today. Link layer hardware (NICs) is often designed to run at "**wire speed**" — handling frames as fast as they arrive. Overruns are then handled by higher layers, not the link layer.

3. Error Detection and Correction

Communication channels exhibit a wide range of error characteristics: **optical fiber** in telecommunications has tiny error rates where transmission errors are rare; **wireless links** and aging local loops have error rates orders of magnitude larger where errors are the norm and cannot be avoided at reasonable cost. **Conclusion:** Transmission errors are here to stay. Network designers must learn to deal with them.

Two Basic Strategies

Table 3-1: Error Handling Strategies

Strategy	Mechanism	Term
Error-correcting codes	Include enough redundancy for receiver to deduce what the transmitted data must have been	FEC (Forward Error Correction)
Error-detecting codes	Include only enough redundancy to detect that an error occurred; receiver requests retransmission	—

Ecological niches: Highly reliable channels (e.g., fiber): cheaper to use error-detecting codes + occasional retransmission. Noisy channels (e.g., wireless): better to add redundancy so receiver can figure out the original block; FEC is preferred because retransmissions are just as likely to be in error as the first transmission.

Error Models

Model 1: Isolated Single-Bit Errors — Caused by extreme values of thermal noise overwhelming the signal briefly. Isolated, occasional single-bit flips.

Model 2: Burst Errors — Errors come in bursts rather than singly. Caused by physical processes: deep fade on wireless channel, transient electrical interference on wired channel.

Trade-offs of Burst Errors

Advantage: Data sent in blocks. With block size 1000 bits and error rate 0.001/bit: independent errors mean most blocks contain an error; bursts of 100 mean only one block in 100 affected on average.

Disadvantage: Much harder to correct than isolated errors when they do occur.

Model 3: Erasure Channels — Location of error is known (physical layer received analog signal far from expected value and declared bit lost). **Easier to correct:** even if the bit value is lost, we know which bit is in error. However, erasures are not always available.

3.1 Error-Correcting Codes

Code Fundamentals

A frame consists of: **m data (message) bits**, **r redundant (check) bits**, with total length $n = m + r$.

Table 3-2: Code Classifications

Property	Definition
Block code	r check bits computed solely as a function of the m data bits with which they are associated (like a lookup table)
Systematic code	m data bits sent directly, along with check bits (not encoded before sending)

Linear code r check bits computed as a linear function of m data bits (XOR or modulo 2 addition); encoding via matrix multiplication or simple logic circuits

Notation: An **(n, m) code** — n -bit unit containing data and check bits is an **n-bit codeword**.

Code rate: fraction of codeword carrying non-redundant information = m/n . Noisy channels: $\sim 1/2$ (half redundant). High-quality channels: close to 1 (small number of check bits).

Hamming Distance

Definition

Given two codewords, the **Hamming distance** is the number of bit positions in which they differ. **Computation:** XOR the two codewords and count the 1 bits in the result.

Example

```
10001001
10110001
-----
00111000 → 3 bits differ → Hamming distance = 3
```

Significance: If two codewords are Hamming distance d apart, it requires d **single-bit errors** to convert one into the other.

Hamming distance of a complete code: The smallest Hamming distance between any two legal codewords in the code.

Sparseness property: With r check bits, only fraction $2^m/2^n = 1/2^r$ of possible messages are legal codewords. This sparseness allows the receiver to detect and correct errors.

Error-Handling Properties by Hamming Distance

Table 3-3: Error Handling Capabilities

Capability	Required Distance	Reasoning
Detect d errors	$d + 1$	d single-bit errors cannot change a valid codeword into another valid codeword; receiver sees illegal codeword

Correct d errors	$2d + 1$	Legal codewords are so far apart that even with d changes, original codeword is still closer than any other; uniquely determinable
-----------------------	----------	--

Example

Code with four valid codewords: 0000000000, 0000011111, 1111100000, 1111111111.

Hamming distance = 5. Can correct double errors or detect quadruple errors.

Hamming Codes

Design Requirement

Code with m message bits and r check bits that allows all **single errors** to be corrected.

Counting argument: Each of the 2^m legal messages has n illegal codewords at distance 1 (formed by inverting each of the n bits). Each legal message requires $n + 1$ dedicated bit patterns.

Total bit patterns available: 2^n . Requirement:

$$(n + 1) \cdot 2^m \leq 2^n$$

Using $n = m + r$:

$$(m + r + 1) \leq 2^r \quad (\text{Equation 3-1})$$

This gives a **lower limit** on check bits needed to correct single errors.

Hamming code construction (Hamming, 1950):

- Bits numbered consecutively starting with bit 1 at left end.
- **Check bits:** positions that are powers of 2 (1, 2, 4, 8, 16, ...)
- **Data bits:** remaining positions (3, 5, 6, 7, 9, ...)

Parity computation: Each check bit forces the modulo 2 sum (parity) of some collection of bits (including itself) to be even (or odd). A bit may be included in several check bit computations.

Rule for check bit coverage: To see which check bits data bit in position k contributes to, rewrite k as a sum of powers of 2. Example: $11 = 1 + 2 + 8$; $29 = 1 + 4 + 8 + 16$. Bit is checked by just those check bits in its expansion.

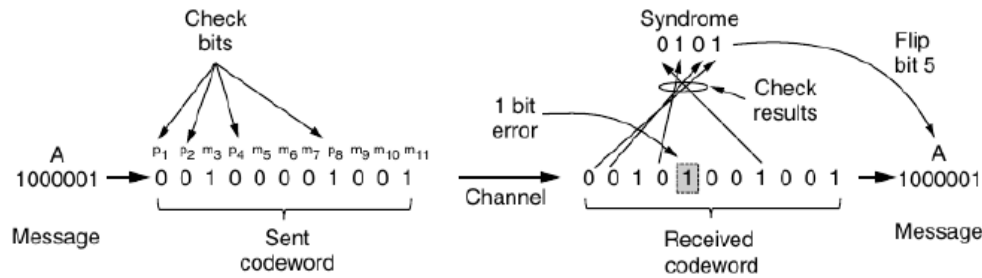


Figure 3-6. Example of an (11, 7) Hamming code correcting a single-bit error.

Figure 3-1: Example of an (11, 7) Hamming code correcting a single-bit error. Message A (ASCII 1000001) is encoded, sent, corrupted at bit 5, detected via syndrome 0101 = 5, and corrected.

(11, 7) Hamming Code Example

Message A (ASCII): 1000001

Position:	1	2	3	4	5	6	7	8	9	10	11
	p1	p2	m1	p4	m2	m3	m4	p8	m5	m6	m7
	0	0	1	0	0	0	0	1	0	0	1

Sent codeword: 00100001001. Hamming distance of 3 → can **correct single errors** or **detect double errors**.

Decoding process:

1. Receiver redoes check bit computations including received check bit values → **check results**.
2. If all check results are zero (for even parity): codeword accepted as valid.
3. If check results not all zero: error detected.

Error syndrome: The set of check results forms the error syndrome, used to pinpoint and correct the error. Example: Check results for $k = 8, 4, 2, 1$: 0, 1, 0, 1. Syndrome: $0101 = 4 + 1 = 5$. By design: **5th bit is in error**. Flip incorrect bit, discard check bits → correct message (ASCII "A").

Convolutional Codes

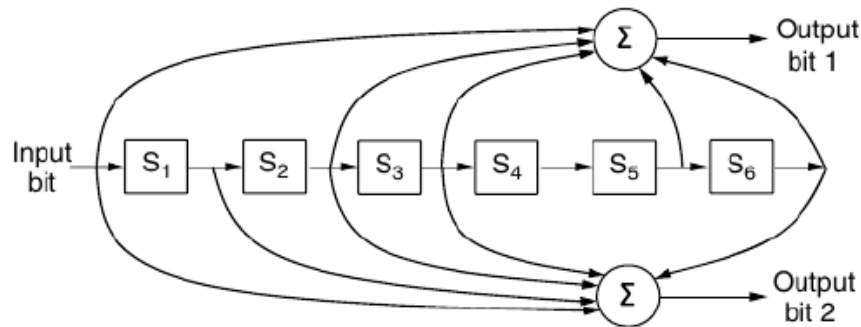


Figure 3-7. The NASA binary convolutional code used in 802.11.

Figure 3-2: The NASA binary convolutional code used in 802.11 (rate $r = 1/2$, constraint length $k = 7$).

Key difference from block codes: Not a block code. Encoder processes a sequence of input bits and generates a sequence of output bits. No natural message size or encoding boundary.

Memory property: Output depends on current and previous input bits. The number of previous bits on which output depends is the **constraint length**.

Specification: In terms of **rate** and **constraint length**. **Deployment:** Widely used in GSM mobile phone system, satellite communications, 802.11.

NASA binary convolutional code (Fig. 3-2): Rate $r = 1/2$, constraint length $k = 7$. First used for Voyager space missions (1977). Reused in 802.11.

- Each input bit produces **two output bits**.
- Output bits are XOR sums of the input and internal state.
- Binary, linear convolutional code.
- Code rate: $1/2$ (1 input $\rightarrow$ 2 output).
- **Not systematic** (none of output bits is simply the input bit).

3.2 Error-Detecting Codes

Usage context: Error-correcting codes are widely used on wireless links (notoriously noisy). Error-detecting codes are usually more efficient over fiber or high-quality copper for occasional errors.

Three Error-Detecting Codes

All are **linear, systematic block codes**:

1. **Parity**
2. **Checksums**
3. **Cyclic Redundancy Checks (CRCs)**

1. Parity

Mechanism: Single parity bit appended to data. Chosen so that the number of 1 bits in the codeword is even (or odd). **Computation:** Equivalent to modulo 2 sum or XOR of data bits.

Parity Example

Data: 1011010

Even parity: 10110100 (total 1s is even)

Odd parity: 10110101 (total 1s is odd)

Properties: Distance of 2 → can **detect single-bit errors**.

Efficiency Comparison

Channel: isolated errors, error rate 10^{-6} per bit. Block size: 1000 bits.

Table 3-4: Overhead Comparison per Megabit

Approach	Overhead per megabit
Hamming correction (Eq. 3-1: 10 check bits)	10,000 bits
Single parity + retransmission	~2,001 bits

Parity calculation: one parity bit per 1000-bit block. One block in 1000 has error → retransmit 1001 bits. Total: ~2001 bits overhead per megabit.

Burst Error Vulnerability

Single parity bit can only reliably detect single-bit errors. Long burst error: detection probability only **0.5** (unacceptable).

Interleaving

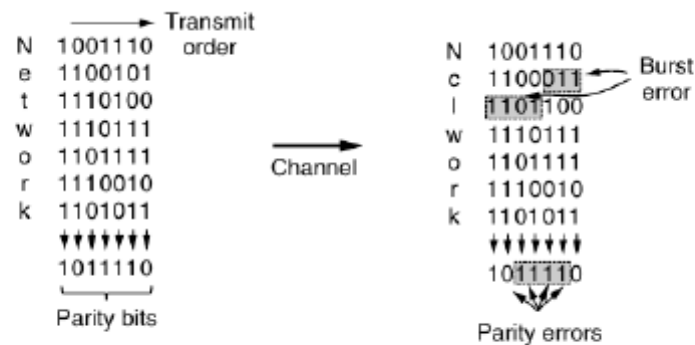


Figure 3-8. Interleaving of parity bits to detect a burst error.

Figure 3-3: Interleaving of parity bits to detect a burst error: 7×7 matrix, transmit order vs. burst error spread across columns causing parity errors.

Technique: Compute parity bits over data in different order than transmission order. Compute parity bit for each of n columns. Send all data bits as k rows (top to bottom, left to right). Last row: n parity bits.

Interleaving principle: Converts code detecting isolated errors into code detecting burst errors. Burst error of length n : bits in error spread across different columns. At most 1 bit in each of n columns affected. Parity bits on those columns detect the error.

Properties: Uses n parity bits on blocks of kn data bits. Detects single burst error of length $\leq n$. Burst of length $n + 1$: passes undetected if first and last bits inverted, others correct. Long burst or multiple shorter bursts: probability of bad block accepted = 2^{-n} .

2. Checksums

Definition: "Checksum" often means any group of check bits associated with a message. A group of parity bits is one example.

Stronger checksums: Based on running sum of data bits. Usually at end of message, as complement of sum function. **Verification:** Sum entire received codeword (data + checksum). If result is zero: no error detected.

Example: 16-bit Internet checksum (IP protocol): Sum of message bits divided into 16-bit words. Operates on words rather than bits. Detects errors that leave parity unchanged (e.g., flipping lowest-order bit in two different words).

One's complement arithmetic: Negative number = bitwise complement of positive counterpart. On two's complement computers: one's complement sum $\equiv$ sum modulo $2^{16} +$

overflow added back into low-order bits.

Weaknesses of Internet Checksum

- Assumption that frames contain random bits is wrong — real data patterns are non-random.
- Undetected errors more common than previously thought (Partridge et al., 1995).
- Does not detect deletion/addition of zero data.
- Does not detect swapping parts of message.
- Weak protection against message splices (parts of two packets combined).

Stronger alternative — Fletcher's checksum (Fletcher, 1982): Includes **positional component**: adds product of data and its position to running sum. Stronger detection of changes in data position.

3. Cyclic Redundancy Checks (CRCs)

Also known as **polynomial codes**. The strongest and most widely used error-detecting code at the link layer.

Polynomial Representation

- Bit string treated as polynomial with coefficients 0 and 1.
- k -bit frame = coefficient list for polynomial with k terms, from x^{k-1} to x^0 .
- High-order (leftmost) bit = coefficient of x^{k-1} .

Example

$$110001 \text{ (6 bits)} = 1 \cdot x^5 + 1 \cdot x^4 + 0 \cdot x^3 + 0 \cdot x^2 + 0 \cdot x^1 + 1 \cdot x^0$$

Polynomial arithmetic: Modulo 2 (no carries, no borrows). Addition and subtraction identical to XOR.

CRC Algorithm

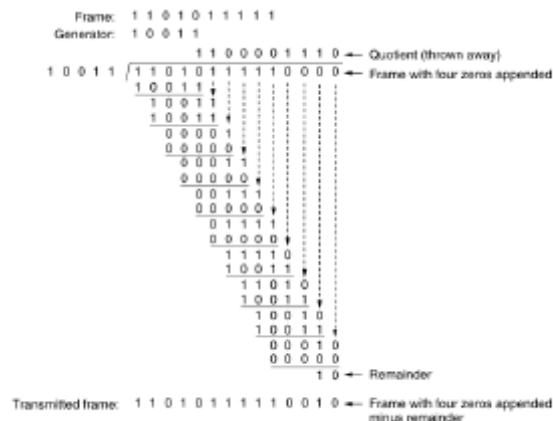


Figure 3-9. Example calculation of the CRC.

Figure 3-4: Example calculation of the CRC: long division of frame 1101011111 by generator 10011 ($x^4 + x + 1$), showing quotient, remainder, and transmitted frame.

Prerequisite: Sender and receiver agree on **generator polynomial** $G(x)$ in advance. Both high- and low-order bits of $G(x)$ must be 1. Frame to be checksummed must be longer than generator polynomial.

Goal: Append CRC such that polynomial of checksummed frame is **divisible by** $G(x)$.

Algorithm:

1. Let $r = \text{degree of } G(x)$. Append r **zero bits** to low-order end of frame. Frame now has $m + r$ bits, corresponding to polynomial $x^r \cdot M(x)$.
2. Divide bit string of $G(x)$ into bit string of $x^r \cdot M(x)$, using **modulo 2 division**.
3. **Subtract the remainder** (always r or fewer bits) from bit string of $x^r \cdot M(x)$ using modulo 2 subtraction.
4. Result = $T(x)$, the checksummed frame to be transmitted.

CRC Example (Fig. 3-4)

Frame: 1101011111

Generator $G(x) = x^4 + x + 1 \rightarrow$ bit string: 10011 (degree $r = 4$)

Append 4 zeros: 1101011111 0000

Divide by 10011 $\rightarrow$ Quotient: 1100001110 (discarded), Remainder: 10

Transmitted frame: 1101011111 **0010** (frame with 4 zeros minus remainder)

Why it works: $T(x)$ is divisible by $G(x)$. In any division, if you diminish dividend by remainder, result is divisible by divisor.

Error Detection Analysis

Received frame: $T(x) + E(x)$, where $E(x)$ has 1s at positions of inverted bits.

Receiver computes: $[T(x) + E(x)]/G(x) = E(x)/G(x)$ (since $T(x)/G(x) = 0$)

Errors slip by iff $E(x)$ contains $G(x)$ as a factor.

Table 3-5: CRC Error Detection Capabilities

Error Type	Detection
Single-bit error ($E(x) = x^i$)	Always detected if $G(x)$ has ≥ 2 terms
Two isolated single-bit errors	Detected if $G(x)$ not divisible by x and $G(x)$ does not divide $x^k + 1$ for k up to max frame length
Odd number of bit errors	Always detected if $x + 1$ is a factor of $G(x)$
Burst error of length $\leq r$	Always detected if $G(x)$ contains x^0 term
Burst error of length $r + 1$	Undetected iff burst identical to $G(x)$. Probability: $1/2^{r-1}$
Burst error $> r + 1$	Probability of undetected bad frame: $1/2^r$

Standard CRC Polynomials**IEEE 802 / Ethernet standard:**

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x^1 + 1$$

Properties: Detects all bursts of length ≤ 32 . Detects all bursts affecting odd number of bits. Used widely since 1980s.

Hardware implementation: Easy to compute and verify with simple **shift register circuits** (Peterson and Brown, 1961). Nearly always used in practice.

Deployment: Dozens of networking standards include various CRCs — virtually all LANs (Ethernet, 802.11) and point-to-point links (packets over SONET).

4. Carrier Sense Multiple Access Protocols

Motivation

With **slotted ALOHA**, the best channel utilization achievable is $1/e \approx 0.368$. This low result is expected because stations transmit at will without knowing what others are doing, leading to many collisions. In **LANs**, stations can often detect what other stations are doing and adapt behavior accordingly. These networks achieve **much better utilization** than $1/e$.

Carrier Sense Protocols

Protocols in which stations listen for a carrier (a transmission) and act accordingly.

4.1 CSMA Variants

1-Persistent CSMA

Mechanism:

1. Station has data to send → first **listens to the channel**.
2. If channel is **idle** → station sends its data immediately.
3. If channel is **busy** → station waits until it becomes idle, then transmits.
4. If a **collision occurs** → station waits a random amount of time and starts over.

Name origin: "1-persistent" because the station transmits with **probability 1** when it finds the channel idle.

Collision Scenarios (despite carrier sensing)

1. **Simultaneous ready stations:** Two stations become ready during a third station's transmission. Both wait politely until it ends, then begin transmitting simultaneously → collision.
2. **Propagation delay effect:** Just after a station begins sending, another station may become ready and sense the channel. If the first station's signal has not yet reached the second, the latter senses an idle channel and also begins sending → collision.

Propagation delay significance: Depends on the **bandwidth-delay product** of the channel. In most LANs: propagation delay is small, only a tiny fraction of a frame fits on the channel

→ small collision chance. Larger bandwidth-delay product → more important effect → worse protocol performance.

Nonpersistent CSMA

Design goal: Consciously less greedy than 1-persistent CSMA.

Mechanism:

1. Station senses channel when it wants to send.
2. If channel is **idle** → station begins transmitting.
3. If channel is **busy** → station **does not continually sense** it to seize immediately upon the end of previous transmission.
4. Instead, it **waits a random period of time** and repeats the algorithm.

Trade-off: Better channel utilization but **longer delays** than 1-persistent CSMA.

p-Persistent CSMA

Applicability: Slotted channels.

Mechanism:

1. Station becomes ready → senses channel.
2. If channel is **idle**: transmits with **probability** p , defers until next slot with **probability** $q = 1 - p$.
3. If next slot is also idle: repeat transmit/defer with probabilities p and q .
4. Process repeats until: frame has been transmitted, OR another station has begun transmitting (treat as collision: wait random time and start again).
5. If channel initially sensed as **busy** → wait until next slot and apply above algorithm.

Deployment: IEEE 802.11 uses a refinement of p-persistent CSMA.

Performance Comparison

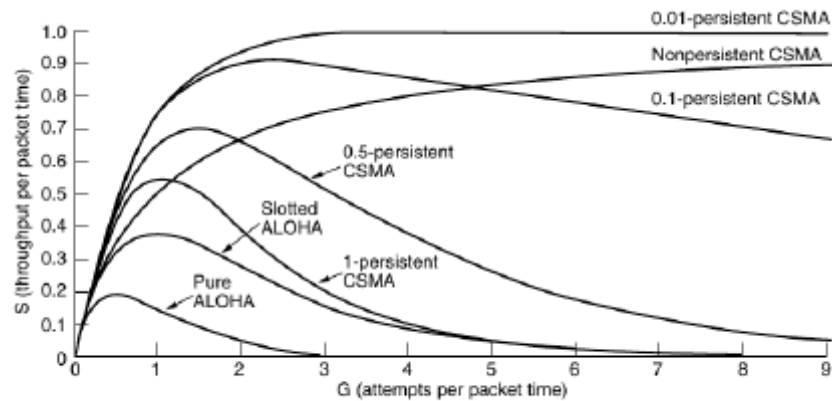


Figure 4-4. Comparison of the channel utilization versus load for various random access protocols.

Figure 4-1: Comparison of channel utilization S versus offered load G for various random access protocols.

Table 4-1: Throughput Comparison of Random Access Protocols

Protocol	Peak Throughput	Behavior at High Load
Pure ALOHA	~ 0.18	Collapses quickly
Slotted ALOHA	~ 0.37 ($1/e$)	Collapses after peak
1-persistent CSMA	~ 0.5	Degrades steadily
0.5-persistent CSMA	~ 0.7	Moderate degradation
0.1-persistent CSMA	~ 0.85	Slow degradation
0.01-persistent CSMA	~ 0.95	Very slow degradation
Nonpersistent CSMA	~ 0.9	Maintains high throughput

Key observations: All CSMA variants outperform ALOHA. Lower persistence (p) $\rightarrow$ higher peak throughput and better high-load behavior. Nonpersistent CSMA achieves near-optimal throughput. Trade-off: lower persistence $\rightarrow$ higher delays.

4.2 CSMA/CD (CSMA with Collision Detection)

Core Improvement

If two stations sense channel idle and begin transmitting simultaneously, their signals still collide. **CSMA/CD** adds the ability to **quickly detect collisions** and **abruptly stop transmitting** (rather than finishing the frame), since garbled frames are irretrievable anyway. This saves time and bandwidth. **Basis of classic Ethernet LAN.**

Collision detection as analog process: Station hardware must **listen to the channel while transmitting**. If received signal differs from transmitted signal → collision is occurring. **Implications:** Received signal must not be tiny compared to transmitted signal (difficult for wireless: received signals may be 1,000,000× weaker). Modulation must allow collision detection.

Conceptual Model

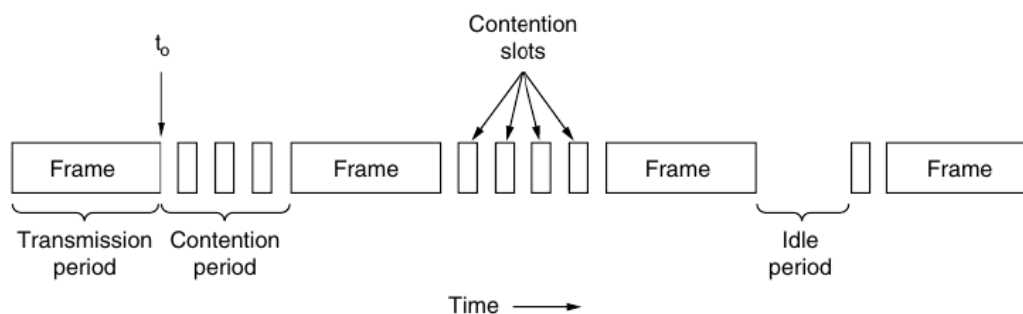


Figure 4-5. CSMA/CD can be in contention, transmission, or idle state.

Figure 4-2: CSMA/CD can be in contention, transmission, or idle state. Alternating transmission periods, contention periods (with slots), and idle periods.

CSMA/CD operates in three states:

- **Transmission periods:** Stations successfully transmitting frames.
- **Contention periods:** Multiple stations attempting to transmit, collisions may occur.
- **Idle periods:** All stations quiet (lack of work).

Contention Period Analysis

Critical question: How long until two colliding stations realize they have collided?

Minimum detection time: Time for signal to propagate from one station to the other.

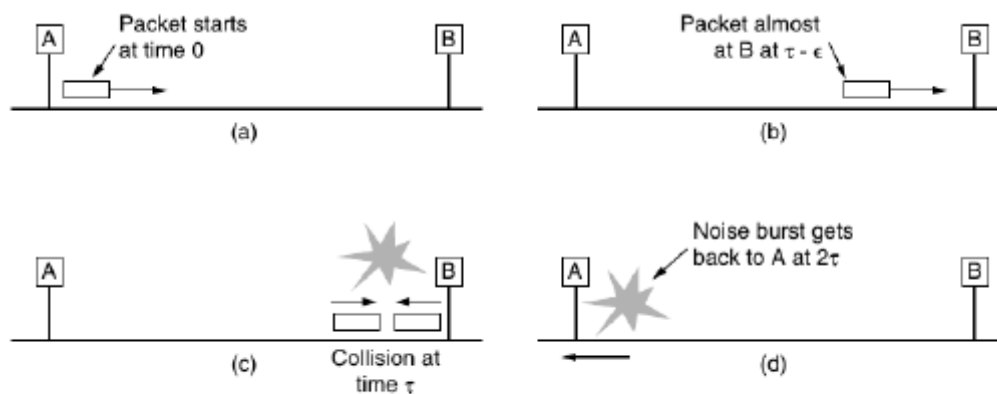


Figure 4-15. Collision detection can take as long as 2τ .

Figure 4-3: Collision detection can take as long as 2τ . (a) Packet starts at A at time 0. (b) Packet almost at B at $\tau - \epsilon$. (c) Collision at B at time τ . (d) Noise burst gets back to A at 2τ .

Table 4-2: Worst-Case Collision Detection Timing

Time	Event
0	Station A begins transmitting
$\tau - \epsilon$	Station B (farthest station) begins transmitting just before A's signal arrives
τ	Collision occurs at B
$2\tau - \epsilon$	Noise burst from collision reaches back to A

Worst-Case Conclusion

A station cannot be sure it has seized the channel until it has transmitted for 2τ without hearing a collision.

CSMA/CD as slotted ALOHA: Can think of CSMA/CD contention as slotted ALOHA with **slot width** = 2τ . **Practical example:** 1-km coaxial cable $\rightarrow \tau \approx 5 \mu\text{sec}$. **Performance improvement over slotted ALOHA:** Slots in which only one station transmits (channel seized) are followed by the **rest of a frame**. Greatly improves performance if frame time $\gg$ propagation time.

5. Classic Ethernet MAC Sublayer Protocol

5.1 Frame Format

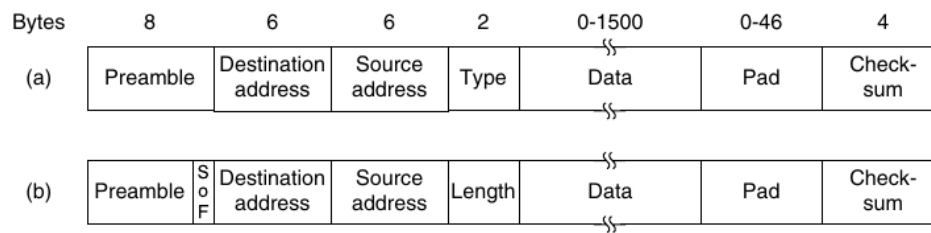


Figure 4-14. Frame formats. (a) Ethernet (DIX). (b) IEEE 802.3.

Figure 5-1: Frame formats: (a) Ethernet (DIX) with Type field. (b) IEEE 802.3 with Length field and SOF delimiter.

(a) Ethernet (DIX — Digital, Intel, Xerox)

Field	Size (bytes)	Description
Preamble	8	Synchronization pattern
Destination address	6	Recipient MAC address
Source address	6	Sender MAC address
Type	2	Indicates upper-layer protocol
Data	0–1500	Payload
Pad	0–46	Padding to meet minimum frame size
Checksum	4	32-bit CRC

(b) IEEE 802.3

Field	Size (bytes)	Description
Preamble	7	Synchronization
SOF (Start of Frame delimiter)	1	Marks frame start
Destination address	6	Recipient MAC address
Source address	6	Sender MAC address
Length	2	Frame length (≤ 1500)

Data	0–1500	Payload (includes LLC header)
Pad	0–46	Padding
Checksum	4	32-bit CRC

Field Details

Preamble

- **8 bytes** (DIX) or **7 bytes + SOF** (802.3).
- Each byte contains bit pattern 10101010.
- Exception: last byte (SOF delimiter) has last 2 bits set to 11.
- **Manchester encoding** produces 10-MHz square wave for 6.4 μ sec.
- Allows receiver's clock to synchronize with sender's.
- Last two 1 bits tell receiver that the rest of the frame is about to start.

Destination and Source Addresses

Size: 6 bytes (48 bits) each. **Destination address** — **first transmitted bit:** 0 → ordinary (unicast) address; 1 → group address.

Table 5-1: Address Types

Type	Description
Unicast	Ordinary address; frame delivered to single station
Multicast (group address)	Multiple stations listen to single address; all stations in group receive frame
Broadcast	Special address of all 1s; accepted by all stations on network

Source addresses — **globally unique:** Assigned centrally by IEEE. **OUI (Organizationally Unique Identifier):** First 3 bytes assigned by IEEE indicate manufacturer. Last 3 bytes assigned by manufacturer, programmed into NIC before sale.

Type vs. Length Field

Ethernet (DIX) — **Type field:** Tells receiver what to do with the frame. Multiple network-layer protocols may be in use simultaneously. Example: Type code 0x0800 = data contains IPv4 packet.

IEEE 802.3 — **Length field:** Carries frame length. No way for receiver to figure out what to do with incoming frame. **Solution:** Added **LLC (Logical Link Control)** header within data.

Resolution (1997)

By 1997, so much DIX Ethernet hardware/software in use that few wanted to repackage Type/Length fields. IEEE compromise: **both ways are fine. Rule:** Values $\leq 0x600$ (1536) → interpreted as **Length**; values $> 0x600$ → interpreted as **Type**.

Data Field

- **Maximum:** 1500 bytes.
- Chosen somewhat arbitrarily when Ethernet standard was finalized.
- Based on transceiver RAM requirements (RAM was expensive in 1978).

Minimum Frame Length and Pad Field

Minimum frame length: 64 bytes (from destination address to checksum, inclusive).

Why Minimum Length is Required

1. **Distinguish valid frames from garbage:** When transceiver detects collision, it truncates current frame. Stray bits and pieces of frames appear on cable constantly. Valid frames must be ≥ 64 bytes to distinguish from garbage.
2. **Prevent undetected collisions on short frames** (Fig. 5-2): If a station transmits a very short frame, transmission may complete **before** the noise burst gets back at 2τ . Sender incorrectly concludes frame was successfully sent.

Solution: All frames must take **more than** 2τ to send, so transmission is still in progress when noise burst returns.

Calculation for 10-Mbps LAN

Maximum length: 2500 meters + four repeaters. Worst-case round-trip time (including repeaters): nearly **50 μsec** . At 10 Mbps: 1 bit = 100 nsec. Minimum guaranteed frame: **500 bits**. With safety margin: rounded up to **512 bits = 64 bytes**.

Checksum

- **32-bit CRC.**
- Defined by the same generator polynomial from Section 3 (used in PPP, ADSL, and other links).
- Error-detecting code: frame **dropped** if error detected.
- No retransmission at data link layer; recovery left to higher layers.

5.2 CSMA/CD with Binary Exponential Backoff

Classic Ethernet uses **1-persistent CSMA/CD**:

- Stations sense medium when they have a frame to send.
- Send frame as soon as medium becomes idle.
- Monitor channel for collisions during transmission.
- If collision: abort with short **jam signal**, retransmit after random interval.

Binary Exponential Backoff Algorithm

Slot Definition

After collision, time divided into discrete slots of length = worst-case round-trip propagation time (2τ). For Ethernet: **512 bit times = 51.2 μ sec**.

Table 5-2: Binary Exponential Backoff

Collision Number (i)	Randomization Interval	Slots to Wait
1	[0, 1]	0 or 1
2	[0, 3]	0, 1, 2, or 3
3	[0, 7]	0 to 7
...	...	...
i	$[0, 2^i - 1]$	0 to $2^i - 1$
≥ 10	[0, 1023]	frozen at max 1023
16	—	Failure reported to computer

Design rationale:

- **Low delay when few stations collide:** Small randomization interval (0 or 1 slots) after first collision.
- **Reasonable resolution when many stations collide:** Interval grows exponentially, ensuring collisions resolve in reasonable time.
- **Truncation at 1023:** Prevents bound from growing too large.

No Acknowledgements in CSMA/CD

If no collision, sender assumes frame was probably successfully delivered. **No acknowledgements** in CSMA/CD or Ethernet. Appropriate for wired/optical fiber with low error rates. Errors detected by CRC, recovered by higher layers. Wireless channels (more errors) use acknowledgements.

6. Data Link Control Protocols

Traditionally, **four protocols** have been defined for the data-link layer to deal with flow and error control:

Table 6-1: Data-Link Layer Protocols

Protocol	Status at Data-Link Layer
Simple	Still used; discussed here
Stop-and-Wait	Still used; discussed here
Go-Back-N	Disappeared from data-link layer; full detail in transport layer
Selective-Repeat	Disappeared from data-link layer; full detail in transport layer

Finite State Machine (FSM) Representation

Figure 11.6 Connectionless and connection-oriented service represented as FSMs

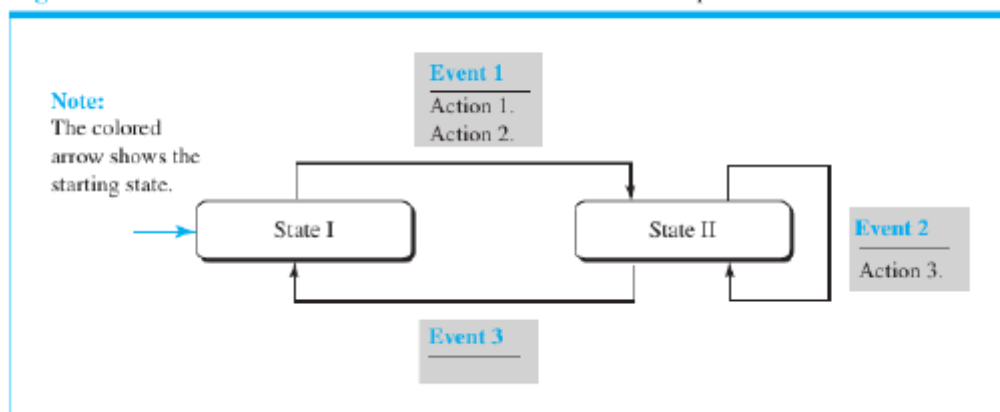


Figure 6-1: FSM notation example: Rounded-corner rectangles = states, colored text = events, regular black text = actions, horizontal line separates event from actions, arrow = movement to next state.

The behavior of data-link-layer protocols is best shown as a **finite state machine (FSM)**. A machine with a finite number of states. The machine is always in one state until an event occurs. **Each event is associated with two reactions:** (1) A list (possibly empty) of **actions** to be performed, and (2) Determination of the **next state** (which may be the same as the current state). **One state must be defined as the initial state.**

6.1 Simple Protocol

Characteristics

- **No flow control.**
- **No error control.**
- Assumption: receiver can immediately handle any frame it receives (receiver can never be overwhelmed).

Figure 11.7 Simple protocol

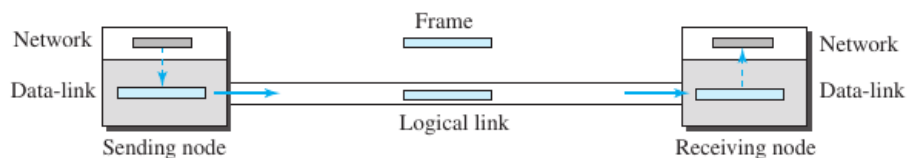


Figure 6-2: Simple protocol layout: sending node → logical link → receiving node.

Operation:

- **Sender data-link layer:** Gets packet from network layer → makes a frame → sends the frame.
- **Receiver data-link layer:** Receives frame from link → extracts packet → delivers packet to network layer.

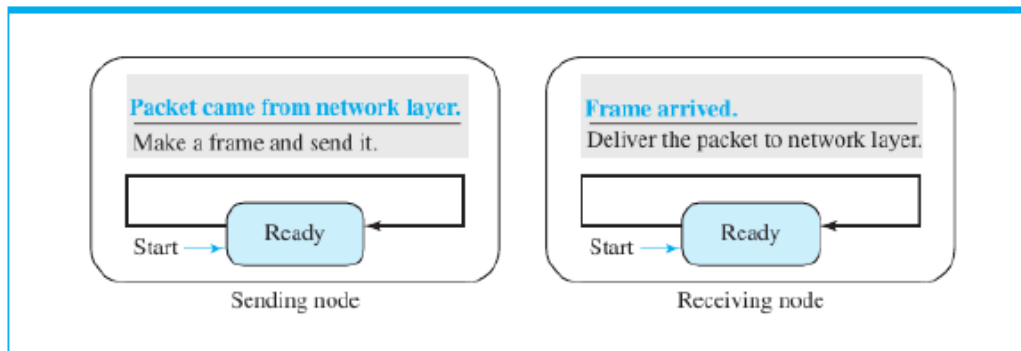
Figure 11.8 FSMs for the simple protocol

Figure 6-3: FSMs for simple protocol: sender (Ready → send frame) and receiver (Ready → deliver packet).

FSMs for Simple Protocol

Sending node: One state: Ready. Event: "Packet came from network layer." Action: "Make a frame and send it." Transition: Self-loop (remains in Ready).

Receiving node: One state: Ready. Event: "Frame arrived." Action: "Deliver the packet to network layer." Transition: Self-loop (remains in Ready).

Example 11.2

Figure 11.9 shows an example of communication using this protocol. It is very simple. The sender sends frames one after another without even thinking about the receiver.

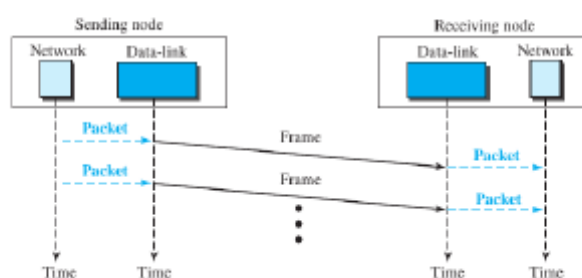
Figure 11.9 Flow diagram for Example 11.2

Figure 6-4: Flow diagram for Simple protocol: continuous unidirectional frame transmission without flow/error control.

6.2 Stop-and-Wait Protocol

Characteristics

- Uses **both flow control and error control**.

- Primitive version discussed here; sophisticated version with sliding windows in transport layer.

Figure 11.10 *Stop-and-Wait protocol*

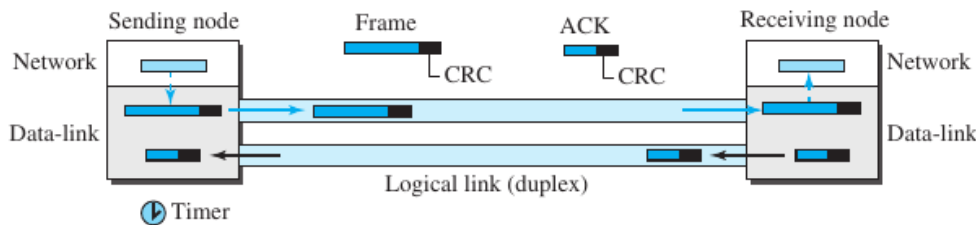


Figure 6-5: *Stop-and-Wait protocol layout: duplex logical link with Frame + CRC, ACK + CRC, and timer.*

Mechanism

- **Logical link is duplex.**
- Sender sends **one frame at a time** and waits for an **acknowledgment (ACK)** before sending the next.
- **CRC** added to each data frame for error detection.
- Only **one frame and one ACK** can be in the channels at any time.

Error detection: When a frame arrives at receiver, CRC is checked. If CRC is **incorrect**: frame is **corrupted** and **silently discarded**. The **silence of the receiver** signals the sender that a frame was either corrupted or lost.

Timer operation: Every time sender sends a frame, it **starts a timer**. If ACK arrives **before timer expires**: timer is stopped, sender sends next frame (if ready). If **timer expires**: sender **resends previous frame** (assuming frame was lost or corrupted). Sender must **keep a copy** of the frame until its ACK arrives.

Figure 11.11 *FSM for the Stop-and-Wait protocol*

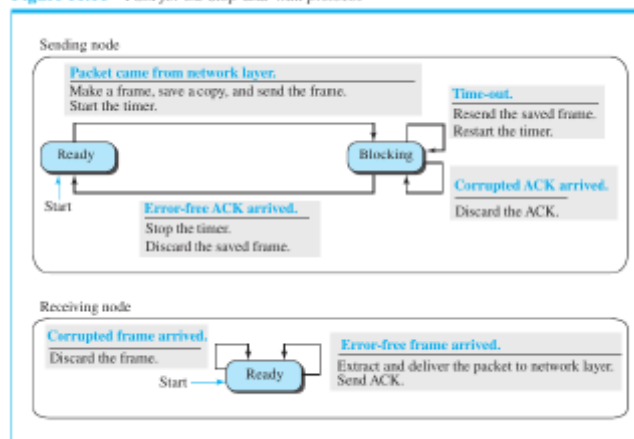


Figure 6-6: *FSMs for Stop-and-Wait: sender (Ready/Blocking states) and receiver (Ready state).*

FSMs for Stop-and-Wait

Sending Node States:

State	Description
Ready	Waiting for packet from network layer
Blocking	Waiting for ACK or time-out

Ready State: Event: "Packet came from network layer." Actions: (1) Make a frame, (2) Save a copy of the frame, (3) Start the timer, (4) Send the frame. Next state: Blocking.

Blocking State — three possible events:

Event	Actions	Next State
Time-out	Resend the saved frame; Restart the timer	Blocking (self-loop)
Corrupted ACK arrived	Discard the ACK	Blocking (self-loop)
Error-free ACK arrived	Stop the timer; Discard the saved frame	Ready

Receiving Node: Always in Ready state.

Event	Actions	Next State
Error-free frame arrived	Extract and deliver packet to network layer; Send ACK	Ready (self-loop)
Corrupted frame arrived	Discard the frame	Ready (self-loop)

Sequence and Acknowledgment Numbers

Figure 11.13 Flow diagram for Example 11.4

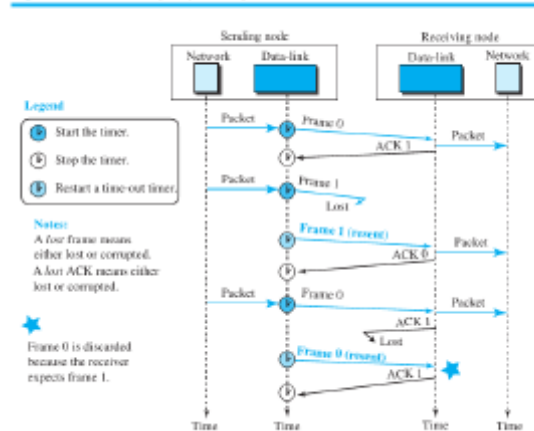


Figure 6-7: Flow diagram for Stop-and-Wait with sequence and acknowledgment numbers preventing duplicate delivery.

The Duplicate Problem

Scenario: First frame sent and acknowledged successfully. Second frame sent but **lost** → after time-out, resent. Third frame sent and acknowledged, but the **ACK is lost** → frame is resent. The network layer at the receiver receives **two copies of the third packet**. Duplicate packets are incorrect.

Solution: Add **sequence numbers** to data frames and **acknowledgment numbers** to ACK frames.

Numbering scheme (simple 1-bit alternating):

- **Sequence numbers:** 0, 1, 0, 1, 0, 1, ...
- **Acknowledgment numbers:** 1, 0, 1, 0, 1, 0, ...
- Sequence numbers start with **0**.
- Acknowledgment numbers start with **1**.
- An acknowledgment number always defines the **sequence number of the next frame to receive**.

Key insight (Fig. 6-7): The receiver uses sequence numbers to detect duplicates. When the resent Frame 0 arrives, the receiver was expecting Frame 1 (acknowledgment number 1 indicates next expected frame is 1), so it discards the duplicate Frame 0 and retransmits ACK 1.

Piggybacking

The Simple and Stop-and-Wait protocols are designed for **unidirectional communication**.

Piggybacking concept: Data in one direction is combined with (piggybacked on) the acknowledgment in the other direction. Piggybacking is **not a common practice** at the data-link layer because it makes communication significantly more complicated.

6.3 HDLC (High-level Data Link Control)

Overview

- **Bit-oriented protocol** for communication over **point-to-point** and **multipoint** links.
- Implements the **Stop-and-Wait protocol**.
- More of a **theoretical foundation** than a practical protocol today.
- **Basis for other practical protocols:** PPP, Ethernet protocol, Wireless LAN protocols.

Configurations and Transfer Modes

Figure 11.14 Normal response mode

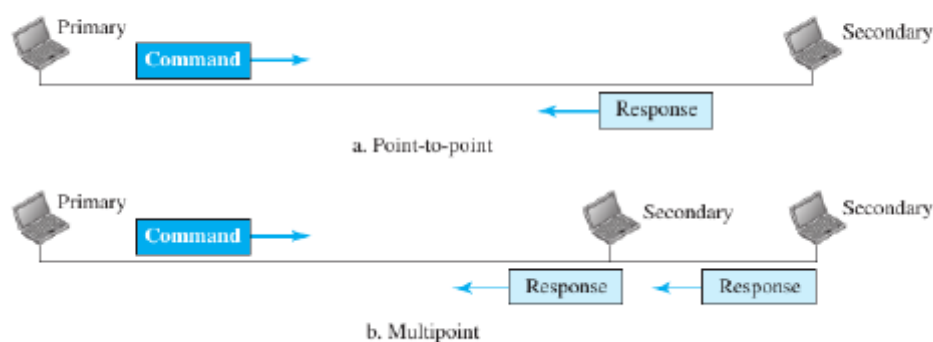


Figure 6-8: Normal Response Mode (NRM): (a) point-to-point and (b) multipoint configurations. One primary station and multiple secondary stations.

1. Normal Response Mode (NRM)

- **Configuration: Unbalanced** — one primary station and multiple secondary stations.
- Primary station: can send **commands**.
- Secondary stations: can only send **responses**.

- **Usage:** Both point-to-point and multipoint links.

Figure 11.15 Asynchronous balanced mode



Figure 6-9: Asynchronous Balanced Mode (ABM): peer-to-peer combined stations. This is the common mode used today.

2. Asynchronous Balanced Mode (ABM)

- **Configuration: Balanced** — link is point-to-point.
- Each station functions as both **primary and secondary** (peers).
- Either station can initiate communication.
- This is the **common mode used today**.

Framing

Figure 11.16 HDLC frames

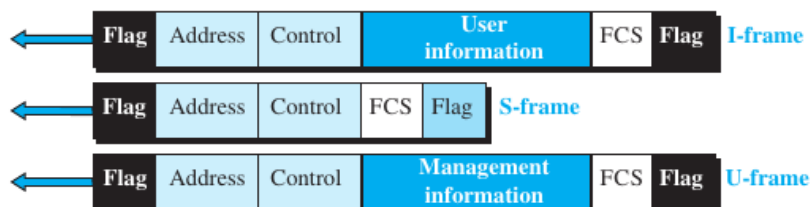


Figure 6-10: HDLC frame formats: I-frame (with user info), S-frame (no info), U-frame (with management info).

HDLC defines **three types of frames**:

Frame Type	Purpose	User Data	Control Info
I-frame (Information)	Carry user data and control info (piggybacking)	Yes	Yes
S-frame (Supervisory)	Transport control information only	No	Yes

U-frame (Unnumbered)	System management	No (management info only)	Yes
-----------------------------	-------------------	---------------------------	-----

Each frame contains up to **six fields**: Flag | Address | Control | Information | FCS | Flag

Table 6-2: HDLC Frame Fields

Field	Description
Flag	Synchronization pattern 01111110; identifies both beginning and end of frame
Address	Address of secondary station. Primary-created frame: to-address . Secondary-created frame: from-address
Control	1 or 2 bytes for flow and error control. Format depends on frame type
Information	User's data from network layer or management information
FCS	Frame Check Sequence; contains 2- or 4-byte CRC

Control Field Formats

Figure 11.17 Control field format for the different frame types

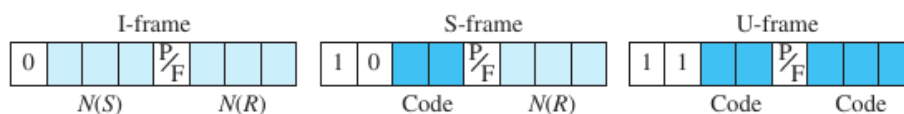


Figure 6-11: Control field formats for I-frame ($0 + N(S) + P/F + N(R)$), S-frame ($10 + \text{Code} + P/F + N(R)$), and U-frame ($11 + \text{Prefix} + P/F + \text{Suffix}$).

I-Frame Control Field

Subfield	Bits	Description
First bit	1	Always 0 → identifies I-frame
N(S)	3	Sequence number of frame (0–7)
P/F	1	Poll/Final bit (Poll when primary → secondary, Final when secondary → primary)
N(R)	3	Acknowledgment number (for piggybacking)

S-Frame Control Field

Code	Type	Description
00	RR — Receive Ready	Acknowledges receipt of safe frame(s)
10	RNR — Receive Not Ready	Acknowledges receipt + announces receiver is busy (congestion control)
01	REJ — Reject	NAK frame for Go-Back-N ARQ
11	SREJ — Selective Reject	NAK frame for Selective Repeat ARQ

Note: S-frames do **not** have information fields.

U-Frame Control Field

- First 2 bits: 11 → identifies U-frame.
- 5 bits total (2-bit prefix + 3-bit suffix) → up to **32 different types of U-frames**.
- Purpose: Exchange session management and control information.
- Contains information field for **system management information** (not user data).

Figure 11.18 Example of connection and disconnection

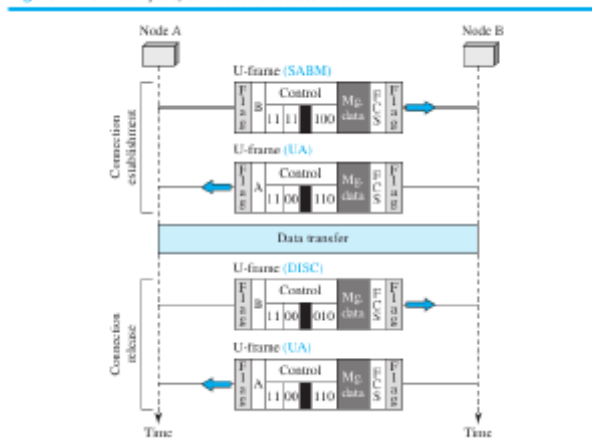


Figure 6-12: Example of connection establishment (SABM/UA) and connection release (DISC/UA) using U-frames.

Connection Establishment and Release

Table 6-3: Connection Phases

Phase	Direction	Frame	Details
-------	-----------	-------	---------

Establishment	A → B	U-frame (SABM)	Set Asynchronous Balanced Mode
	B → A	U-frame (UA)	Unnumbered Acknowledgment
Data Transfer	Data transfer occurs between the two nodes		
Release	A → B	U-frame (DISC)	Disconnect
	B → A	U-frame (UA)	Unnumbered Acknowledgment

Figure 11.19 Example of piggybacking with and without error

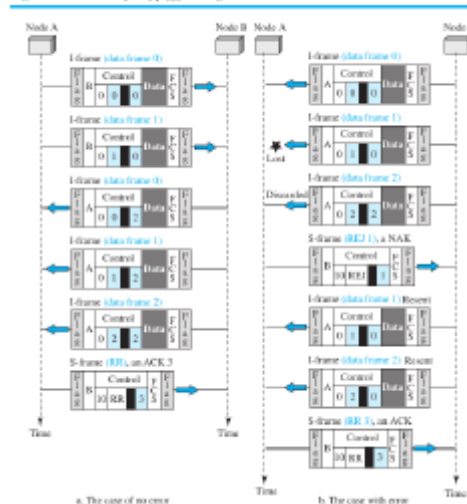


Figure 6-13: Example of piggybacking: (a) no error case with data frames carrying piggybacked acknowledgments. (b) error case with REJ/NAK recovery.

6.4 Point-to-Point Protocol (PPP)

Overview

- One of the most common protocols for **point-to-point access**.
- Millions of Internet users connect home computers to ISP servers via PPP.
- Typical setup: traditional modem over telephone line (physical layer) + PPP at data-link layer.
- **Most common** point-to-point data-link protocol.

Services Provided by PPP

Service	Description
Frame format definition	Defines format of frames exchanged between devices

Link negotiation	Defines how two devices negotiate link establishment and data exchange
Multi-network-layer support	Accepts payloads from several network layers (not only IP)
Authentication	Provided but optional
Network address configuration	Particularly useful for home users needing temporary network address

Services NOT Provided by PPP

Service	Status	Details
Flow control	Not provided	Sender can send several frames one after another
Error control	Minimal	CRC field used to detect errors. Corrupted frame silently discarded; upper-layer handles recovery
Multipoint addressing	Not provided	No sophisticated addressing mechanism

Framing

Figure 11.20 PPP frame format

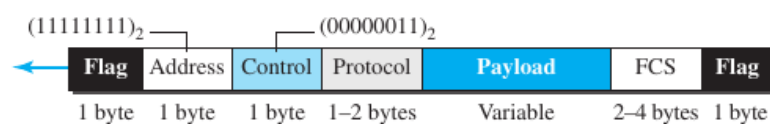


Figure 6-14: PPP frame format: Flag, Address, Control, Protocol, Payload, FCS, Flag.

PPP uses a **character-oriented (byte-oriented) frame**.

Table 6-4: PPP Frame Fields

Field	Size	Value / Description
Flag	1 byte	01111110 — marks start and end of frame
Address	1 byte	Constant 11111111 (broadcast address)
Control	1 byte	Constant 00000011 — imitates unnumbered frames in HDLC
Protocol	1-2 bytes	Defines what is carried in data field

Payload	Variable (max 1500 bytes)	Carries user data or other information. Byte-stuffed if flag pattern appears
FCS	2 or 4 bytes	Standard CRC

Byte Stuffing in PPP

Since PPP is **byte-oriented**, the flag byte must be escaped when it appears in the data section.

Aspect	Detail
Escape byte	01111101
Mechanism	Whenever flag-like pattern (01111110) appears in data, escape byte is stuffed before it
Escape byte itself	Must also be stuffed with another escape byte if it appears in data

Transition Phases

Figure 11.21 Transition phases

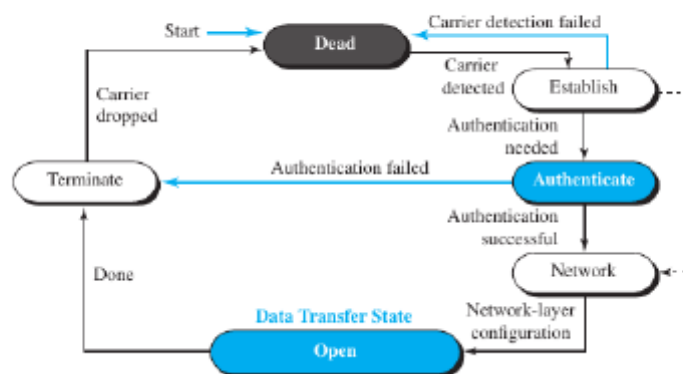


Figure 6-15: PPP transition phases: Dead → Establish → Authenticate → Network → Open → Terminate → Dead.

Table 6-5: PPP Phase Descriptions

Phase	Description
Dead	Initial state. No active carrier at physical layer; line is quiet
Establish	One node starts communication. Options are negotiated between parties
Authenticate	(Optional) If authentication is needed. If fails → goes to Terminate

Network	Network-layer configuration performed
Open	Data Transfer State. Exchange of data packets begins
Terminate	Connection termination requested. Returns to Dead when carrier is dropped

7. Practice Questions & Answers

How to Use This Section

This section contains exam-style questions covering both theoretical concepts and numerical problems. Review these after studying the notes to test your understanding. Numerical problems include step-by-step solutions.

7.1 Theoretical Questions

Q1. What are the two principal parts of the telephone system from an engineering perspective? Explain each.

Answer: The two principal parts are: (1) **Outside plant** — consists of local loops and trunks, physically located outside switching offices. (2) **Inside plant** — consists of switches located inside switching offices. The outside plant handles the physical connections to subscribers and between offices, while the inside plant handles the switching and routing of calls.

Q2. Explain the fundamental difference between circuit switching and packet switching with respect to path establishment.

Answer: In **circuit switching**, a dedicated physical path must be established from sender to receiver before any data can be transmitted. This setup takes time (easily 10+ seconds for long-distance calls). In **packet switching**, there is no need to set up a dedicated path in advance. Packets are sent as soon as they are available, and each packet is routed independently through the network.

Q3. Why is packet switching more fault-tolerant than circuit switching?

Answer: In circuit switching, if a switch fails, **all circuits using it are terminated** because each call depends on a specific dedicated path. In packet switching, packets can be **dynamically rerouted around dead switches** since there is no fixed path. Each packet can take a different route, so the failure of one switch does not prevent packets from reaching their destination.

Q4. What is the fundamental trade-off between circuit switching and packet switching?

Answer: The fundamental trade-off is: **guaranteed service + wasted resources** (circuit switching) versus **no guarantee + no waste** (packet switching). Circuit switching provides guaranteed bandwidth and constant delay but wastes resources when the connection is idle. Packet switching uses bandwidth efficiently but cannot guarantee delay or throughput.

Q5. List and briefly explain the three types of services provided by the data link layer to the network layer.

Answer: (1) **Unacknowledged Connectionless Service:** Frames sent independently without acknowledgements. No logical connection. Lost frames not recovered at data link layer. Used in Ethernet. (2) **Acknowledged Connectionless Service:** No logical connections, but each frame is individually acknowledged. Lost frames can be retransmitted. Used in 802.11 (WiFi). (3) **Acknowledged Connection-Oriented Service:** Connection established before data transfer. Each frame numbered. Guarantees exactly-once, in-order delivery. Used for satellite channels and long-distance circuits.

Q6. Why is providing acknowledgements at the data link layer considered an "optimization" rather than a requirement?

Answer: The network layer could always implement its own acknowledgement and retransmission mechanism. However, the data link layer knows **link-specific parameters** (maximum frame length, propagation delays) that the network layer does not. If a large packet is broken into 10 frames and 2 are lost, network-layer retransmission of the entire packet would be very slow. Data-link-layer frame acknowledgement and retransmission correct errors more directly and quickly, making it an optimization for efficiency.

Q7. Explain the four methods of framing used in the data link layer.

Answer: (1) **Byte Count:** A field in the header specifies the number of bytes in the frame. Vulnerable to errors in the count field. (2) **Flag Bytes with Byte Stuffing:** Special flag bytes delimit frames. If flag pattern appears in data, an escape byte (ESC) is inserted before it. (3) **Flag Bits with Bit Stuffing:** Flag pattern 01111110 delimits frames. After five consecutive 1s in data, a 0 is stuffed. Used in HDLC, USB. (4) **Physical Layer Coding Violations:** Uses reserved signal patterns in physical encoding (e.g., 4B/5B) to mark frame boundaries. No stuffing needed.

Q8. Why is byte count framing rarely used by itself?

Answer: The byte count field can be **garbled by transmission errors**. If the count becomes corrupted (e.g., due to a bit flip), the destination loses synchronization and cannot locate the correct start of the next frame. Even if the checksum detects the error, the receiver doesn't know how many bytes to skip to find the next frame. Retransmission requests are ineffective because the destination cannot tell where the next frame starts.

Q9. Explain the difference between error-correcting codes (FEC) and error-detecting codes.

Answer: Error-correcting codes (FEC — Forward Error Correction) include enough redundant information for the receiver to deduce what the original transmitted data must have been. The receiver can correct errors without requesting retransmission. Preferred on noisy channels (e.g., wireless) where retransmissions are likely to also be corrupted. **Error-detecting codes** include only enough redundancy to detect that an error occurred. The receiver then requests retransmission. More efficient on reliable channels (e.g., fiber) where errors are rare.

Q10. What is Hamming distance and why is it important?

Answer: The **Hamming distance** between two codewords is the number of bit positions in which they differ. It is computed by XORing the two codewords and counting the 1 bits. It is important because: (1) To **detect d errors**, a code needs Hamming distance of at least $d + 1$. (2) To **correct d errors**, a code needs Hamming distance of at least $2d + 1$. The minimum Hamming distance between any two valid codewords determines the error-handling capability of the entire code.

Q11. Explain the relationship between the number of check bits r and message bits m in Hamming codes (Equation 3-1).

Answer: The Hamming code requirement is: $(m + r + 1) \leq 2^r$. This ensures there are enough bit patterns to represent all 2^m legal messages plus their $n = m + r$ single-error variants (distance-1 codewords), plus the valid codeword itself. Each of the 2^m messages requires $n + 1 = m + r + 1$ dedicated patterns, and the total available patterns is $2^n = 2^{m+r}$.

Q12. Compare parity, checksum, and CRC as error-detecting mechanisms.

Answer: Parity: Single bit appended to make total 1s even/odd. Distance of 2. Detects single-bit errors only. Vulnerable to burst errors (50% detection probability). **Checksum:** Based on running sum of data bits (e.g., 16-bit Internet checksum). Detects errors that leave parity unchanged. Weaker than CRC; doesn't detect zero additions or message swaps. **CRC (Cyclic Redundancy Check):** Strongest and most widely used. Based on polynomial division. Can detect all burst errors up to length r , all odd-numbered bit errors (if $x + 1$ is a factor), and has controllable undetected error probability of $1/2^r$.

Q13. What is the "hidden terminal problem" and how does CSMA/CD address it differently from wireless networks?

Answer: The hidden terminal problem occurs when two stations cannot sense each other's transmissions but can both reach a common receiver, causing collisions. **CSMA/CD (wired)** addresses this by having stations **listen while transmitting** — if the received signal differs from the transmitted signal, a collision is detected and transmission is aborted. This works because wired signals have predictable strength. **In wireless**, received signals may be 1,000,000× weaker than transmitted signals, making collision detection impractical. Wireless uses CSMA/CA (Collision Avoidance) instead, with RTS/CTS handshaking.

Q14. Explain binary exponential backoff in CSMA/CD. Why is it designed this way?

Answer: After a collision, the randomization interval grows exponentially: $[0, 2^i - 1]$ slots after the i -th collision, capped at $[0, 1023]$ after 10 collisions, and failure reported after 16 collisions. **Rationale:** (1) **Low delay when few stations collide:** Small interval (0 or 1 slots) after first collision. (2) **Reasonable resolution when many collide:** Exponential growth ensures collisions resolve in reasonable time. (3) **Truncation at 1023** prevents the bound from growing too large and causing excessive delays.

Q15. Why is there a minimum frame size of 64 bytes in Ethernet?

Answer: Two reasons: (1) **Distinguish valid frames from garbage:** When transceiver detects collision, it truncates current frame. Stray bits appear on cable constantly. Valid frames must be ≥ 64 bytes to be distinguishable from garbage. (2) **Prevent undetected collisions on short frames:** A station must still be transmitting when the collision noise burst returns at time 2τ . If a frame is too short, transmission completes before the noise burst returns, and the sender incorrectly assumes success. At 10 Mbps with worst-case round-trip of $\sim 50 \mu\text{sec}$, minimum frame = 512 bits = 64 bytes.

Q16. Explain the difference between Ethernet (DIX) and IEEE 802.3 frame formats.

Answer: Both have the same overall structure but differ in: (1) **Preamble:** DIX uses 8 bytes; 802.3 uses 7 bytes + 1 SOF delimiter byte. (2) **Type/Length field:** DIX uses a **Type** field (values > 1536) indicating the upper-layer protocol (e.g., 0x0800 = IPv4). IEEE 802.3 uses a **Length** field (≤ 1500) carrying frame length. This required adding an LLC header inside the data field to identify the protocol. (3) The 1997 compromise: values $\leq 0x600$ = Length; values $> 0x600$ = Type.

Q17. What is piggybacking and why is it not commonly used at the data link layer?

Answer: **Piggybacking** is the technique of combining data in one direction with acknowledgements in the other direction within the same frame (e.g., when node A sends data to B, it also includes ACK for data received from B). It is **not commonly used at the data link layer** because it makes communication significantly more complicated. The data link layer typically handles unidirectional traffic efficiently, and the added complexity of managing bidirectional data+ack combinations outweighs the benefit for most link-layer scenarios.

Q18. Compare NRM and ABM in HDLC.

Answer: **NRM (Normal Response Mode):** Unbalanced configuration with one primary station and multiple secondary stations. Primary sends commands; secondaries can only send responses. Used for both point-to-point and multipoint links. **ABM (Asynchronous Balanced Mode):** Balanced configuration for point-to-point links. Each station functions as both primary and secondary (peers). Either station can initiate communication. ABM is the **common mode used today**.

Q19. What services does PPP NOT provide compared to HDLC?

Answer: PPP does **not provide**: (1) **Flow control** — sender can send several frames one after another without concern about overwhelming the receiver. (2) **Error control** (beyond detection) — CRC is used to detect errors, but corrupted frames are silently discarded; upper-layer protocol handles recovery. (3) **Multipoint addressing** — no sophisticated addressing for multipoint configurations. The Address field is set to constant 11111111 (broadcast), and the Control field is constant 00000011.

Q20. List the four data-link layer protocols for flow and error control and their current status.

Answer: (1) **Simple Protocol**: Still used; no flow or error control. (2) **Stop-and-Wait Protocol**: Still used; implements both flow and error control. (3) **Go-Back-N**: Disappeared from data-link layer; full detail in transport layer. (4) **Selective-Repeat**: Disappeared from data-link layer; full detail in transport layer. The first two are discussed at the data link layer because they are needed to understand wired and wireless LANs.

7.2 Numerical Problems

N1. A code has four valid codewords: 0000000000, 0000011111, 1111100000, and 1111111111. What is the Hamming distance of this code? How many errors can it correct? How many can it detect?

Solution:

Step 1: Find Hamming distance between pairs:

- $d(0000000000, 0000011111) = 5$
- $d(0000000000, 1111100000) = 5$
- $d(0000000000, 1111111111) = 10$
- $d(0000011111, 1111100000) = 10$
- $d(0000011111, 1111111111) = 5$
- $d(1111100000, 1111111111) = 5$

Step 2: Minimum Hamming distance = 5.

Step 3: For correction: need $d \geq 2t + 1$, so $5 \geq 2t + 1 \rightarrow t \leq 2$. Can correct **2 errors**.

Step 4: For detection: need $d \geq e + 1$, so $5 \geq e + 1 \rightarrow e \leq 4$. Can detect **4 errors**.

N2. Given a message with $m = 8$ data bits, what is the minimum number of check bits r required for a Hamming code that can correct all single-bit errors?

Solution:

Using the Hamming code inequality: $(m + r + 1) \leq 2^r$

Substitute $m = 8$: $(8 + r + 1) \leq 2^r \rightarrow (9 + r) \leq 2^r$

Test values of r :

- $r = 3$: $9 + 3 = 12 \leq 8$? **No**
- $r = 4$: $9 + 4 = 13 \leq 16$? **Yes**

Minimum check bits required: $r = 4$. This gives a (12, 8) Hamming code.

N3. A channel has an error rate of 10^{-6} per bit. Blocks are 1000 bits. Compare the overhead per megabit for: (a) Hamming code correction (using 10 check bits per block), and (b) single parity bit plus retransmission.

Solution:

(a) Hamming correction:

Overhead = 10 check bits per 1000-bit block.

Blocks per megabit = $10^6 / 1000 = 1000$ blocks.

Total overhead = $10 \times 1000 = \mathbf{10,000}$ bits.

(b) Single parity + retransmission:

Error probability per block = $1000 \times 10^{-6} = 10^{-3}$.

Expected errors per megabit = $1000 \times 10^{-3} = 1$ block in error.

Overhead per error = 1 parity bit + 1000-bit retransmission = 1001 bits.

Plus parity bits for all blocks = $1 \times 1000 = 1000$ bits.

Total overhead = $1000 + 1001 \approx \mathbf{2,001}$ bits.

Conclusion: Parity + retransmission is about 5× **more efficient** for this reliable channel.

N4. In a 10-Mbps CSMA/CD network, the maximum distance between any two stations is 2.5 km and there are four repeaters. The signal propagation speed is 2×10^8 m/s. Calculate the minimum frame size if the worst-case round-trip time (including repeaters) is 50 μ sec.

Solution:

Step 1: Bit time at 10 Mbps = $1/10^7 = 100$ nsec = 0.1 μ sec.

Step 2: Minimum frame bits = Worst-case RTT / Bit time
 $= 50 \mu\text{sec} / 0.1 \mu\text{sec} = 500$ bits.

Step 3: With safety margin, round up to nearest power of 2: **512 bits = 64 bytes.**

Step 4: Verify propagation time:

$$\tau = 2500 \text{ m} / (2 \times 10^8 \text{ m/s}) = 12.5 \mu\text{sec one way.}$$

Worst-case collision detection = $2\tau = 25 \mu\text{sec}$ (plus repeater delays = 50 μ sec total).

N5. A CSMA/CD network uses binary exponential backoff. After the 5th collision, what is the range of slots a station must wait? What is the maximum wait after 12 collisions?

Solution:

Step 1: For the i -th collision, randomization interval is $[0, 2^i - 1]$.

Step 2: After **5th collision**:

$$\text{Interval} = [0, 2^5 - 1] = [0, 31].$$

Station waits a random number of slots from **0 to 31**.

Step 3: After **12th collision**:

Since $i \geq 10$, the interval is **frozen at** $[0, 1023]$.

Station waits a random number of slots from **0 to 1023**.

Step 4: After **16 collisions**, failure is reported to the computer.

N6. An Ethernet frame carries 1000 bytes of data. Calculate the total frame size including all overhead (use DIX Ethernet format). What is the efficiency?

Solution:

Step 1: DIX Ethernet overhead per frame:

- Preamble: 8 bytes
- Destination address: 6 bytes
- Source address: 6 bytes
- Type: 2 bytes
- Pad: 0 bytes (data $\geq$ 46 bytes, so no padding needed)
- Checksum (CRC): 4 bytes

Step 2: Total overhead = $8 + 6 + 6 + 2 + 0 + 4 = 26$ bytes.

Step 3: Total frame size = $1000 + 26 = \mathbf{1026}$ bytes.

Step 4: Efficiency = Data / Total = $1000/1026 = \mathbf{97.5\%}$.

N7. A CRC generator polynomial is $G(x) = x^3 + x + 1$ (bit string 1011). The data frame is 1101. Compute the transmitted frame.

Solution:

Step 1: Degree of $G(x) = r = 3$. Append 3 zeros to data: **1101000**.

Step 2: Divide 1101000 by 1011 using modulo 2 division:

```

      1110
1011 | 1101000
      1011
      ----
        1100
        1011
        ----
          1110
          1011
          ----
            1010
            1011
            ----
              0010 ← Remainder = 010

```

Step 3: Transmitted frame = 1101000 + 010 = **1101010** (data + CRC).

Step 4: Verify: 1101010 / 1011 should have remainder 0.

N8. Using interleaving with a 5×5 matrix, compute the parity bits for the data "HELLO" (ASCII: H=1001000, E=1000101, L=1001100, L=1001100, O=1001111). Show how a burst error of 5 consecutive bits is detected.

Solution:

Step 1: Arrange data in 5×5 matrix (using last 5 bits of each 7-bit ASCII):

```

Row 0 (H): 0 0 1 0 0
Row 1 (E): 1 0 1 0 1
Row 2 (L): 1 0 0 1 1
Row 3 (L): 1 0 0 1 1
Row 4 (O): 1 1 1 1 1

```

Step 2: Compute column parity bits (even parity):

Col 0: $0+1+1+1+1 = 4 \rightarrow \text{parity} = 0$

Col 1: $0+0+0+0+1 = 1 \rightarrow \text{parity} = 1$

Col 2: $1+1+0+0+1 = 3 \rightarrow \text{parity} = 1$

Col 3: $0+0+1+1+1 = 3 \rightarrow \text{parity} = 1$

Col 4: $0+1+1+1+1 = 4 \rightarrow \text{parity} = 0$

Parity row: **0 1 1 1 0**

Step 3: Transmit column by column. A burst of 5 bits affects at most 1 bit per column when spread across the 5 rows. Since each column has its own parity bit, the error in each affected column will be detected.

N9. In a Stop-and-Wait protocol, the frame size is 1000 bits, the channel rate is 1 Mbps, and the one-way propagation delay is 10 msec. Calculate the channel utilization (efficiency).

Solution:

Step 1: Transmission time per frame (T_t) = Frame size / Bandwidth

$$T_t = 1000 \text{ bits} / 10^6 \text{ bps} = 1 \text{ msec.}$$

Step 2: Total time per frame cycle = $T_t + 2 \times T_p + T_{ACK}$

(Assume ACK is small, so $T_{ACK} \approx 0$)

$$\text{Total} = 1 + 2(10) + 0 = 21 \text{ msec.}$$

Step 3: Utilization = $T_t / \text{Total cycle time}$

$$= 1/21 = \mathbf{4.76\%}.$$

Note: This low efficiency is the main limitation of Stop-and-Wait protocols on high-delay channels.

N10. In p-persistent CSMA, a station senses the channel idle in a slotted system with $p = 0.3$. What is the probability that the station transmits in the first slot? What is the probability it defers for exactly 3 slots before transmitting?

Solution:

Given: $p = 0.3$, so $q = 1 - p = 0.7$.

Step 1: Probability of transmitting in first slot = $p = 0.3$.

Step 2: Probability of deferring exactly 3 slots then transmitting:

This means: defer (slot 1) $\times$ defer (slot 2) $\times$ defer (slot 3) $\times$ transmit (slot 4).

$$P = q \times q \times q \times p = q^3 \times p = (0.7)^3 \times 0.3$$

$$= 0.343 \times 0.3 = \mathbf{0.1029 \text{ or } 10.29\%}.$$

N11. An HDLC I-frame has sequence number $N(S) = 5$ and acknowledgment number $N(R) = 3$. The P/F bit is set to 1 and the frame is sent from a secondary to a primary station. (a) What does $N(S) = 5$ mean? (b) What does $N(R) = 3$ mean? (c) What does $P/F = 1$ indicate in this context?

Solution:

(a) $N(S) = 5$: This is the **sequence number of the current frame being sent**. It indicates this is the 5th I-frame in the current sequence.

(b) $N(R) = 3$: This is the **acknowledgment number**. It means the sender has received all I-frames up to sequence number 2 and expects to receive frame 3 next from the other station.

(c) $P/F = 1$ (sent by secondary to primary): Since the frame goes from secondary to primary, the P/F bit set to 1 indicates **Final**. This means it is the final frame in the current response from the secondary station.

N12. A Stop-and-Wait protocol uses 1-bit sequence numbers. Show what happens when: (a) Frame 0 is sent and ACK 1 is received, (b) Frame 1 is lost, (c) Frame 0 (resent after timeout) arrives when receiver expects Frame 1.

Solution:

(a) Frame 0 sent, ACK 1 received:

- Sender transmits Frame (seq=0).
- Receiver accepts Frame 0, delivers packet to network layer, sends ACK (ack=1).
- Sender receives ACK 1, stops timer, discards copy of Frame 0.
- Sender is now ready to send Frame (seq=1).

(b) Frame 1 is lost:

- Sender transmits Frame (seq=1), starts timer.
- Frame 1 is lost in transit.
- Receiver does not receive Frame 1, sends nothing.
- Timer expires. Sender resends Frame (seq=1), restarts timer.
- Receiver gets Frame 1, delivers packet, sends ACK (ack=0).

(c) Duplicate Frame 0 arrives:

- Suppose Frame 0 was sent, ACK 1 was lost.
- Timer expires, sender resends Frame (seq=0).
- Receiver has already accepted Frame 0 and is expecting Frame 1 (ack=1 indicates next expected = 1).
- Receiver sees seq=0 but expects seq=1 → **discards as duplicate**, retransmits ACK 1.
- Sender receives ACK 1, proceeds to send Frame (seq=1).

N13. In a CSMA/CD network with slot time = 51.2 μ sec (512 bit times at 10 Mbps), calculate the throughput efficiency if the average frame size is 64 bytes and the average contention period is 2 slots.

Solution:

Step 1: Frame transmission time = $64 \times 8 \text{ bits} / 10 \text{ Mbps} = 512 \text{ bits} / 10^7 = 51.2 \mu\text{sec}$.

Step 2: Average contention period = $2 \times 51.2 = 102.4 \mu\text{sec}$.

Step 3: Total cycle time = Contention + Transmission = $102.4 + 51.2 = 153.6 \mu\text{sec}$.

Step 4: Efficiency = Transmission time / Total cycle time
 $= 51.2 / 153.6 = 1/3 = \mathbf{33.3\%}$.

Note: This is a simplified model. Actual efficiency depends on number of stations, traffic load, and frame size distribution.

N14. For an (11, 7) Hamming code, a received codeword is 00101001001. The check bits are at positions 1, 2, 4, 8. Check bits cover: p1 covers positions 1,3,5,7,9,11; p2 covers 2,3,6,7,10,11; p4 covers 4,5,6,7; p8 covers 8,9,10,11. Determine if an error occurred and correct it.

Solution:

Received: Position 1 2 3 4 5 6 7 8 9 10 11

Bits: 0 0 1 0 1 0 0 1 0 0 1

Step 1: Check p1 (positions 1,3,5,7,9,11): $0+1+1+0+0+1 = 3$ (odd) → **Error in group** → $c1 = 1$

Step 2: Check p2 (positions 2,3,6,7,10,11): $0+1+0+0+0+1 = 2$ (even) → OK → $c2 = 0$

Step 3: Check p4 (positions 4,5,6,7): $0+1+0+0 = 1$ (odd) → **Error in group** → $c4 = 1$

Step 4: Check p8 (positions 8,9,10,11): $1+0+0+1 = 2$ (even) → OK → $c8 = 0$

Step 5: Syndrome = $c8\ c4\ c2\ c1 = 0101$ (binary) = **5 (decimal)**.

Step 6: **Bit 5 is in error**. Flip it: bit 5 changes from 1 to 0.

Step 7: Corrected codeword: 001**0**0001001

Data bits (positions 3,5,6,7,9,10,11): 1 0 0 0 0 0 1 = **1000001** (ASCII "A").

N15. A Pure ALOHA system operates at 100 kbps. The average frame is 1000 bits. Calculate the maximum achievable throughput in kbps.

Solution:

Step 1: Frame transmission time $T_f = 1000 \text{ bits} / 100 \text{ kbps} = 10 \text{ msec}$.

Step 2: For Pure ALOHA, maximum throughput occurs at $G = 0.5$:

$$S_{max} = G \times e^{-2G} = 0.5 \times e^{-1} = 0.5 / 2.718 = 0.1839.$$

Step 3: Maximum throughput = $0.1839 \times 100 \text{ kbps} = \mathbf{18.39 \text{ kbps}}$.

Note: This is the theoretical maximum (~18.4% of channel capacity). For Slotted ALOHA, $S_{max} = 1/e \approx 36.8\%$ at $G = 1$.

N16. In a byte-stuffing scheme using flag byte 01111110 and escape byte 01111101, show the transmitted byte sequence for the data: FLAG, A, ESC, B, FLAG, C.

Solution:

Original data: | FLAG | A | ESC | B | FLAG | C |

Step 1: Flag at start of frame → delimiter (not stuffed).

Step 2: FLAG in data → stuff with ESC: **ESC, FLAG**

Step 3: A → A (no change)

Step 4: ESC in data → stuff with ESC: **ESC, ESC**

Step 5: B → B (no change)

Step 6: FLAG in data → stuff with ESC: **ESC, FLAG**

Step 7: C → C (no change)

Step 8: Flag at end of frame → delimiter (not stuffed).

Transmitted frame:

| FLAG | ESC | FLAG | A | ESC | ESC | B | ESC | FLAG | C | FLAG |